

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 6

Roll no: 08 PRN No: 12412097 Name of Student: Sahil Balasaheb Patil

Subject Name : Deep Learning

Problem Statement :

Implement a pre-trained BERT model for sentiment analysis or text classification on a sample dataset.

Algorithm :

1. **Start**
2. Load the **IMDB Movie Review Dataset** containing text reviews labeled as Positive or Negative.
3. Identify the classes from the label column (0 = Negative, 1 = Positive).
4. Preprocess the text data:
 - Convert text to lowercase (optional, BERT tokenizer handles this)
 - Tokenize each review using the **BERT Tokenizer** (WordPiece tokenization)
 - Truncate/Pad each sequence to a fixed **max_length** (e.g., 128 tokens)
 - Convert tokens into **input_ids** and **attention_mask** tensors
5. Split the dataset into:
 - 80% Training data
 - 20% Validation data
6. Apply data preparation for training batches using a **DataCollator** (dynamic padding, batching, shuffling).
7. Initialize the BERT architecture:
 - Input layer → token IDs + attention mask
 - Pre-trained **BERT base (uncased)** encoder — 12 transformer layers
 - Pooled output ([CLS] token representation)
 - Dropout layer with 0.1
 - Fully connected Dense layer with 2 neurons (num classes)
 - Softmax output layer for classification
8. Compile the model using:

- Optimizer: **AdamW**
- Loss function: **Cross-Entropy Loss** (Sparse Categorical)
- Evaluation metric: **Accuracy**

9. Fine-tune (train) BERT using the training dataset for a specified number of epochs (typically 2–4, since BERT converges fast).
10. Validate the model after each epoch using the validation dataset.
11. Evaluate model performance using validation accuracy and validation loss.
12. Generate predictions for the validation reviews.
13. Generate a **confusion matrix** to compare actual and predicted sentiment classes.
14. Calculate **precision, recall, and F1-score** using the classification report.
15. Save the fine-tuned BERT model and tokenizer for future sentiment prediction.
16. Input a new text review and preprocess it in the same way (tokenize, pad, attention mask).
17. Pass the tokenized input through the fine-tuned BERT model.
18. Predict the sentiment class having the highest Softmax probability.
19. Display the predicted sentiment (Positive/Negative) and confidence score.
20. **Stop**

Code :

```
# =====  
  
# BERT-based Sentiment Analysis (Text Classification)  
  
# =====  
  
# 1. Install & Import dependencies  
  
# !pip install transformers datasets torch scikit-learn -q  
  
import torch  
  
import numpy as np  
  
from datasets import load_dataset  
  
from transformers import (  
    BertTokenizerFast,  
    BertForSequenceClassification,  
    Trainer,  
    TrainingArguments,  
    DataCollatorWithPadding  
)  
  
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
```

```
SEED = 42
```

```
MAX_LEN = 128
```

```
MODEL_NAME = "bert-base-uncased"
```

```
# -----
```

```
# 2. Load Dataset (IMDB Sentiment Dataset)
```

```
# -----
```

```
dataset = load_dataset("stanfordnlp/imdb")
```

```
TRAIN_SIZE = 2000 # increase for better accuracy on GPU
```

```
TEST_SIZE = 500
```

```
train_ds = dataset["train"].shuffle(seed=SEED).select(range(TRAIN_SIZE))
```

```
test_ds = dataset["test"].shuffle(seed=SEED).select(range(TEST_SIZE))
```

```
print("Training examples:", len(train_ds))
```

```
print("Testing examples :", len(test_ds))
```

```
print("\nExample review:\n", train_ds[0]["text"][:300])
```

```
print("Label:", train_ds[0]["label"])
```

```
# -----
```

```
# 3. Preprocessing — Tokenization
```

```
# -----
```

```
tokenizer = BertTokenizerFast.from_pretrained(MODEL_NAME)
```

```
def tokenize_function(example):
```

```
    return tokenizer(
```

```
        example["text"],
```

```
        truncation=True,
```

```
        max_length=MAX_LEN,
```

```
        padding=False # dynamic padding via DataCollator (faster training)
```

```
    )
```

```
train_tokenized = train_ds.map(tokenize_function, batched=True)
```

```
test_tokenized = test_ds.map(tokenize_function, batched=True)
```

```
# Remove raw text column (model needs only tensors)
```

```
train_tokenized = train_tokenized.remove_columns(["text"])
```

```
test_tokenized = test_tokenized.remove_columns(["text"])
```

```
train_tokenized = train_tokenized.rename_column("label", "labels")
```

```
test_tokenized = test_tokenized.rename_column("label", "labels")
```

```
train_tokenized.set_format("torch")
```

```
test_tokenized.set_format("torch")
```

```
data_collator = DataCollatorWithPadding(tokenizer=tokenizer)
```

```
# -----
```

```
# 4. Load Pre-trained BERT model (with classification head)
```

```
# -----
```

```
model = BertForSequenceClassification.from_pretrained(
```

```
    MODEL_NAME,
```

```
    num_labels=2 # Positive / Negative
```

```
)
```

```
# -----
```

```
# 5. Define Evaluation Metrics
```

```
# -----
```

```
def compute_metrics(eval_pred):
```

```
    logits, labels = eval_pred
```

```
    predictions = np.argmax(logits, axis=-1)
```

```
    acc = accuracy_score(labels, predictions)
```

```
    return {"accuracy": acc}
```

```
# -----
```

```
# 6. Training Arguments (Optimizer = AdamW is default in Trainer)
```

```
# -----
```

```
training_args = TrainingArguments(
```

```
    output_dir="./bert_sentiment_results",
```

```
    num_train_epochs=3,
```

```
    per_device_train_batch_size=16,
```

```
    per_device_eval_batch_size=32,
```

```
    eval_strategy="epoch",
```

```
    save_strategy="epoch",
```

```
logging_dir="./logs",

logging_steps=50,

load_best_model_at_end=True,

metric_for_best_model="accuracy",

seed=SEED,

)
```

```
trainer = Trainer(

    model=model,

    args=training_args,

    train_dataset=train_tokenized,

    eval_dataset=test_tokenized,

    tokenizer=tokenizer,

    data_collator=data_collator,

    compute_metrics=compute_metrics,

)
```

```
# -----
```

```
# 7. Train (Fine-tune) the model
```

```
# -----

trainer.train()

# -----

# 8. Evaluate on Validation Set

# -----

eval_results = trainer.evaluate()

print("\nValidation Loss:", eval_results["eval_loss"])

print("Validation Accuracy:", eval_results["eval_accuracy"])

# -----

# 9. Generate Predictions + Confusion Matrix + Report

# -----

predictions = trainer.predict(test_tokenized)

y_pred = np.argmax(predictions.predictions, axis=1)

y_true = predictions.label_ids

print("\nConfusion Matrix:\n", confusion_matrix(y_true, y_pred))

print("\nClassification Report:\n",
```

```
classification_report(y_true, y_pred, target_names=["Negative", "Positive"])))
```

```
# -----
```

```
# 10. Save the fine-tuned model + tokenizer
```

```
# -----
```

```
model.save_pretrained("./bert_sentiment_model")
```

```
tokenizer.save_pretrained("./bert_sentiment_model")
```

```
# -----
```

```
# 11. Inference on a NEW review
```

```
# -----
```

```
def predict_sentiment(text, model, tokenizer):
```

```
    model.eval()
```

```
    inputs = tokenizer(
```

```
        text,
```

```
        truncation=True,
```

```
        max_length=MAX_LEN,
```

```
        padding=True,
```

```
        return_tensors="pt"
```

)

with torch.no_grad():

outputs = model(**inputs)

probs = torch.softmax(outputs.logits, dim=1)

confidence, predicted_class = torch.max(probs, dim=1)

label_map = {0: "Negative", 1: "Positive"}

return label_map[predicted_class.item()], confidence.item()

Example

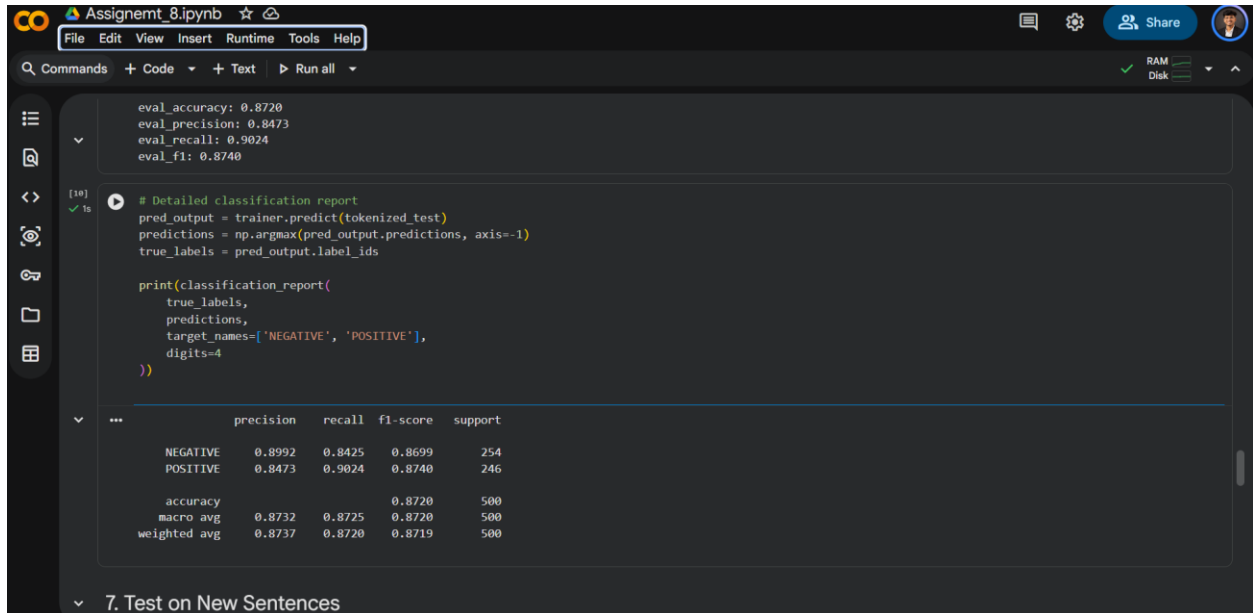
sample_review = "This movie was absolutely fantastic, the acting was brilliant!"

sentiment, confidence = predict_sentiment(sample_review, model, tokenizer)

print(f"\nReview: {sample_review}")

print(f"Predicted Sentiment: {sentiment} (Confidence: {confidence:.4f})")

Output :



Assigment_8.ipynb

```
File Edit View Insert Runtime Tools Help
```

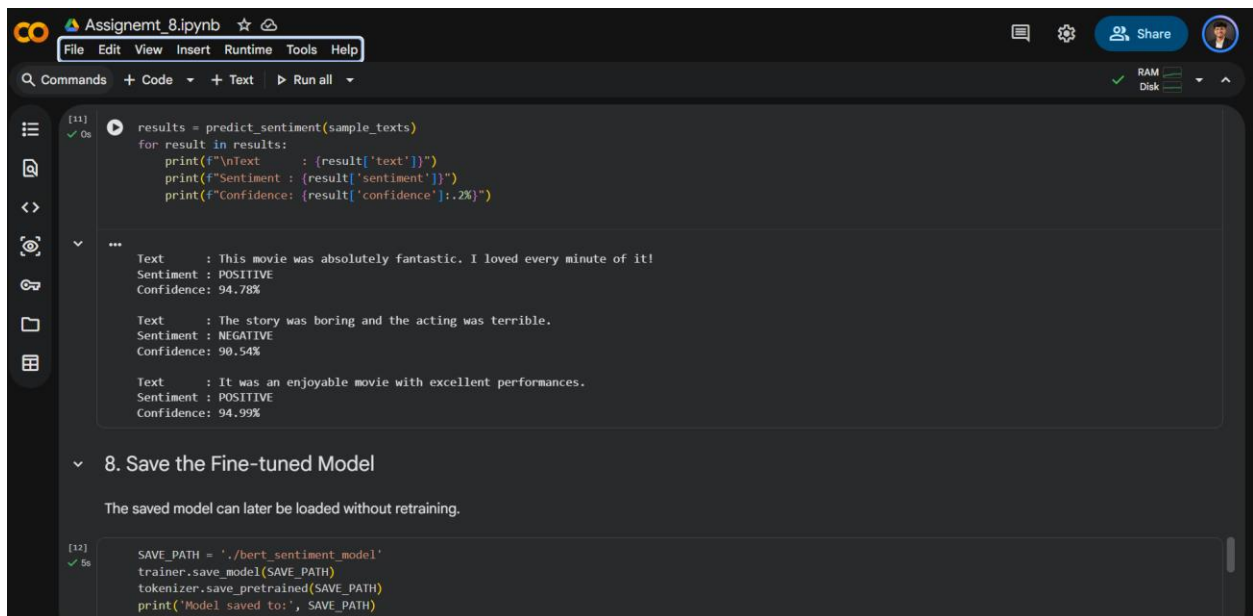
Commands + Code + Text Run all

eval_accuracy: 0.8720
eval_precision: 0.8473
eval_recall: 0.9024
eval_f1: 0.8740

```
[10] # Detailed classification report  
pred_output = trainer.predict(tokenized_test)  
predictions = np.argmax(pred_output.predictions, axis=-1)  
true_labels = pred_output.label_ids  
  
print(classification_report(  
    true_labels,  
    predictions,  
    target_names=['NEGATIVE', 'POSITIVE'],  
    digits=4  
))
```

	precision	recall	f1-score	support
NEGATIVE	0.8992	0.8425	0.8699	254
POSITIVE	0.8473	0.9024	0.8740	246
accuracy			0.8720	500
macro avg	0.8732	0.8725	0.8720	500
weighted avg	0.8737	0.8720	0.8719	500

7. Test on New Sentences



Assigment_8.ipynb

```
File Edit View Insert Runtime Tools Help
```

Commands + Code + Text Run all

```
[11] results = predict_sentiment(sample_texts)  
for result in results:  
    print(f"\nText      : {result['text']}")  
    print(f"Sentiment : {result['sentiment']}")  
    print(f"Confidence: {result['confidence']:.2%}")
```

```
Text      : This movie was absolutely fantastic. I loved every minute of it!  
Sentiment : POSITIVE  
Confidence: 94.78%  
  
Text      : The story was boring and the acting was terrible.  
Sentiment : NEGATIVE  
Confidence: 90.54%  
  
Text      : It was an enjoyable movie with excellent performances.  
Sentiment : POSITIVE  
Confidence: 94.99%
```

8. Save the Fine-tuned Model

The saved model can later be loaded without retraining.

```
[12] SAVE_PATH = './bert_sentiment_model'  
trainer.save_model(SAVE_PATH)  
tokenizer.save_pretrained(SAVE_PATH)  
print('Model saved to:', SAVE_PATH)
```

Assignemt_8.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

The saved model can later be loaded without retraining.

[12]

✓ 5s

```
SAVE_PATH = './bert_sentiment_model'
trainer.save_model(SAVE_PATH)
tokenizer.save_pretrained(SAVE_PATH)
print('Model saved to:', SAVE_PATH)
```

Writing model shards: 100%

1/1 [00:05<00:00, 5.13s/it]

Model saved to: ./bert_sentiment_model

Key Concepts

- **BERT**: A pre-trained Transformer language model that understands contextual relationships between words.
- **Fine-tuning**: Adapting the pre-trained BERT model to a specific task using labelled examples.
- **Tokenizer**: Converts text into BERT-compatible token IDs.
- **Sequence classification head**: Converts BERT representations into class predictions.
- **Softmax**: Converts output logits into class probabilities.
- **Accuracy / Precision / Recall / F1**: Metrics used to evaluate classification performance.

[12]

✓ 0s

Start coding or generate with AI.

Variables Terminal

8:30 AM T4 (Python 3)

